

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Mechatronics Engineering

Course No.: MTE 3104

Course Title: Microcontroller & Interfacing Sessional

Experiment No.: 02

Experiment Name: Blinking LED using ATmega8 and Control Using IR Sensor

Remarks

<u>Submitted By:</u>	<u>Submitted To:</u>
Bayazid Habib Siddikee 3rd year (Odd Semester) Roll:2208053 Department of Mechatronics Engineering, RUET	Prangon Das Assistant Professor, Department of Mechatronics Engineering, RUET

Date of Experiment: 12/1/2026

Date of Submission: 7/4/2026

Experiment No.: 02

Experiment Name: Blinking LED using ATmega8 and Control Using IR Sensor

Objectives:

- To understand the working of the ATmega8 in embedded systems.
- To learn how to program a standalone microcontroller using Arduino Uno as an ISP (In-System Programmer).
- To interface an IR sensor with the ATmega8 and observe its behavior.
- To implement a simple control system where LEDs respond to object detection.
- To gain practical knowledge of digital input/output operations and basic embedded programming.

Required Equipment's:

- Arduino UNO, ATmega8 or ATmega32
- IR sensor, LED's, 1k ohm resistance, jumpers
- Tools: avr-gcc, avr-objcopy, avr-dude or Arduino IDE

Introduction:

The ATmega8 is a widely used 8-bit microcontroller from the AVR family, originally developed by Atmel. It is designed for embedded systems and offers a balanced combination of performance, low power consumption, and ease of use. With features such as 8 KB of flash memory, built-in analog-to-digital converters, timers, and communication interfaces like SPI and USART, the ATmega8 is suitable for a variety of applications including robotics, automation, and sensor-based systems. Its compact design and reliability make it a popular choice for both beginners and professionals in microcontroller-based projects.

To program the ATmega8, an Arduino Uno can be used as an In-System Programmer (ISP). This method allows the microcontroller to be programmed directly within a circuit without removing it. By uploading the ArduinoISP sketch through the Arduino IDE and connecting the appropriate SPI pins between the Arduino Uno and the ATmega8, users can easily burn bootloaders or upload programs. This approach is cost-effective and convenient, especially for prototyping and educational purposes, as it eliminates the need for dedicated programming hardware.

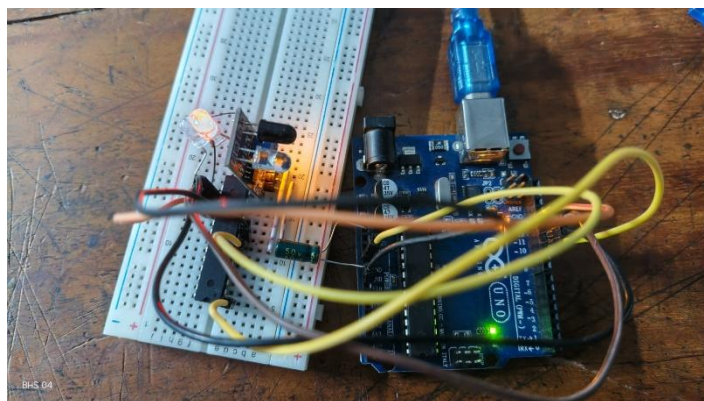


Fig 1: Controlling LED's with the help of IR sensor's output

An Infrared (IR) sensor is an electronic device that detects objects or motion by using infrared radiation, a form of electromagnetic energy beyond the visible spectrum. It typically consists of an IR emitter and a receiver that work together to detect reflected infrared light from nearby objects. When an object comes within range, the emitted infrared light reflects back to the receiver, causing a change in the sensor's output signals.



Fig 2: Arduino UNO Pin Diagram

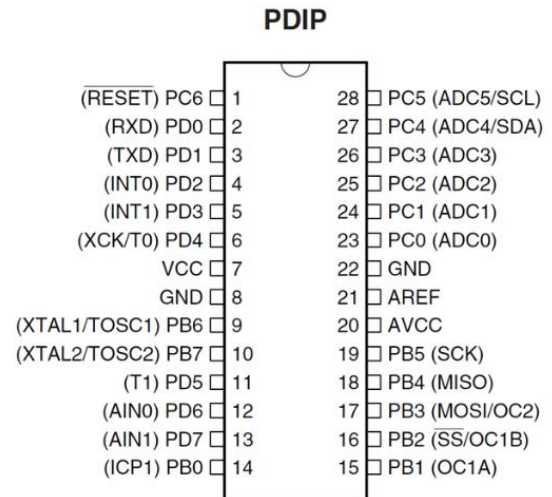


Fig 3: Atmega8 Pin Diagram

Procedure:

Step 1: Connecting Arduino UNO with ATmega8 or Atmega32

Signal	Arduino Uno pin	ATmega8 pin	ATmega32 pin	Description
MOSI	D11 (PB3)	Pin 17 PB3/MOSI	Pin 6 PB5/MOSI	Master out — data to target
MISO	D12 (PB4)	Pin 18 PB4/MISO	Pin 7 PB6/MISO	Master in — data from target
SCK	D13 (PB5)	Pin 19 PB5/SCK	Pin 8 PB7/SCK	SPI clock
RESET	D10 (SS)	Pin 1 PC6/RESET	Pin 9 RESET	Puts target into ISP mode
VCC	5 V	Pins 7 & 20 VCC/AVCC	Pins 10 & 30 VCC/AVCC	Supply voltage (5 V)
GND	GND	Pins 8 & 22 GND/AGND	Pins 11 & 31 GND/AGND	Ground — connect all GND pins

Table 1: Pin diagram of ATmega8, ATmega32 and UNO to Transfer Data Among Them

Step 2: After connecting all pins, I have connected pin 15,16 with two LED's with 1k ohm resistance and then pin14 with IR sensor, there is a 10 micro farad capacitor in between RESET and GND pin so ATmega8 can be reset and upload new files safely.

Step 3: As I'm linux user I needed to install avr-gcc to compile C code of ATmega8, avr-objcopy to make a runnable script on ATmega8 and last avr-dude to upload and burn ATmega8. All the script I have built as [run.sh](#) and the ATmega code as [main.c](#).

Step 4: Run the script and it uploaded the main.c in ATmega8 and LED was on.

Code:

[run.sh](#):

```
avr-gcc -mmcu=atmega8 -DF_CPU=1000000UL -Os -o counter.elf main.c
avr-objcopy -O ihex -R .eeprom counter.elf counter.hex
avrdude -c stk500v1 -p m8 -P /dev/ttyACM0 -b 19200 -B 250 -U
flash:w:counter.hex:i
```

main.c:

```
#include <avr/io.h>
int main(void)
{
    // Set PB1 and PB2 as output (LEDs)
    DDRB |= (1 << PB1) | (1 << PB2);
    // Set PB0 as input (IR sensor)
    DDRB &= ~(1 << PB0);
    while (1) {
        if (PINB & (1 << PB0)) { // Object detected
            PORTB |= (1 << PB1); // LED1 ON
            PORTB &= ~(1 << PB2); // LED2 OFF
        } else { // No object
            PORTB |= (1 << PB2); // LED2 ON
            PORTB &= ~(1 << PB1); // LED1 OFF
        }
    }
}
```

Result:

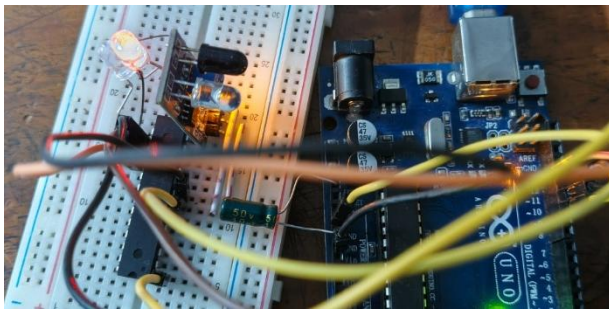


Fig 4: When no object is detected (Color LED)

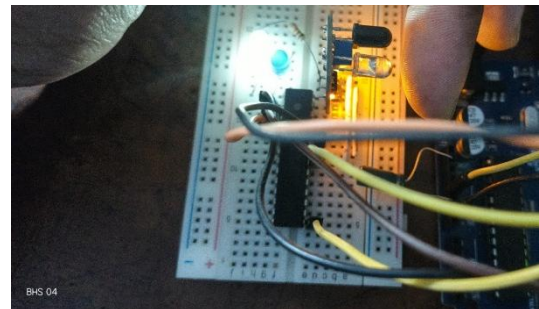


Fig 5: When object is detected (White)

Discussion:

In this experiment, the ATmega8 microcontroller was successfully programmed using Arduino Uno as an ISP, demonstrating a practical and low-cost programming method. The IR sensor was interfaced as a digital input device, providing HIGH or LOW signals depending on object presence. Based on the sensor output, two LEDs were controlled using conditional logic. The system behaved as expected: when an object was detected, LED1 turned ON, indicating detection, while LED2 turned OFF. In the absence of an object, LED2 remained ON continuously, serving as a default state indicator. This confirms proper interfacing of input and output devices and validates the logic implemented in the program.

Conclusion:

The experiment successfully demonstrated the use of Arduino Uno as an ISP to program the ATmega8 microcontroller. It also showed how an IR sensor can be used for object detection and how microcontroller pins can be used to control output devices like LEDs. The results confirm that embedded systems can be effectively built using simple components and basic programming techniques.

Reference:

- Microchip Technology – ATmega8 Datasheet
- AVR Microcontroller Programming Manuals